



Université du Québec à Rimouski

Département de mathématiques, d'informatique et de génie

8INF976 - Sujet spécial en intelligence artificielle

Efficacité et compression

Rapport de lecture

Réalisé par : Pier-Luc Dupéré

Professeur : Mehdi Adda, PhD

Table des matières

1	Quantification	1
2	Élagage (pruning)	2
3	Distillation	3
4	Adaptation efficace (PEFT) LoRA/QLoRA	4

Chapitre 1

Quantification

ous proposons AWQ (Activation-aware Weight Quantization), une méthode adaptée au matériel pour la quantification des poids des LLM en faible précision (low-bit). AWQ repose sur le constat que tous les poids d'un LLM n'ont pas la même importance ; la préservation de seulement 1 pourcentage des poids dits « saillants » permet de réduire considérablement l'erreur de quantification. Pour identifier ces canaux de poids saillants, il convient de se baser sur la distribution des activations plutôt que sur les poids eux-mêmes. Afin d'éviter la quantification à précision mixte (inefficace sur le plan matériel), nous démontrons mathématiquement que l'amplification (mise à l'échelle) des canaux saillants permet de réduire l'erreur de quantification.

La quantification des poids en faible précision pour les LLM peut réduire considérablement l'empreinte mémoire de l'inférence sur appareil, mais elle s'avère complexe. L'entraînement prenant en compte la quantification (QAT) manque d'efficacité en raison de son coût d'entraînement élevé, tandis que la quantification post-entraînement (PTQ) entraîne une dégradation importante de la précision en faible précision. Les travaux les plus proches sont ceux de GPTQ (Frantar et al., 2022), qui utilisent des informations du second ordre pour compenser les erreurs. Toutefois, cette méthode risque de surapprendre l'ensemble de calibration lors de la reconstruction, ce qui altère les caractéristiques apprises pour des domaines hors distribution (Figure 8) ; cela pose problème dans la mesure où les LLM sont des modèles généralistes.

Quantification post-entraînement (PTQ) : INT8, INT4, GGUF, AWQ, GPTQ, SmoothQuant - compromis taille/vitesse/qualité

Quantification pendant l'entraînement (QAT) : bits-and-bytes, QLoRA (4-bit + adapters LoRA).

Chapitre 2

Élagage (pruning)

Élagage (pruning) : structuré vs non structuré, sparseGPT, Wanda, magnitude pruning.

SparseGPT procède en ramenant le problème de l'élagage (ne seule étape) à un ensemble d'instances de régression creuse à très grande échelle. Il résout ensuite ces instances grâce à un nouvel algorithme de résolution approchée pour la régression creuse, suffisamment efficace pour s'exécuter en quelques heures sur les plus grands modèles GPT accessibles au public (175 milliards de paramètres) à l'aide d'un seul GPU

parseGPT offre une précision telle que la perte de performance après élagage est négligeable, et ce sans aucun réglage fin (*fine-tuning*).

L'élagage après entraînement (*Post-Training Pruning*) est un scénario pratique dans lequel on dispose d'un modèle θ bien optimisé et de données de calibration, et où l'on doit obtenir une version compressée de θ

Élagage couche par couche. La compression après entraînement s'effectue généralement en décomposant le problème de compression du modèle complet en sous-problèmes propres à chaque couche.

la qualité de la solution est alors mesurée par l'erreur L2 entre la sortie de la couche non compressée (de poids W) et celle de la couche compressée, pour des entrées X données.

Difficulté de passer à l'échelle pour atteindre plus de 100 milliards de paramètres. Les techniques de post-entraînement existantes ont toutes été conçues pour compresser avec précision des modèles comptant jusqu'à quelques centaines de millions de paramètres.

Chapitre 3

Distillation

Distillation : knowledge distillation classique, distillation de LLM vers petit LLM (TinyLlama, Phi, distil- whisper, etc.)

Chapitre 4

Adaptation efficace (PEFT)

LoRA/QLoRA

QLoRA est une méthode de réglage fin efficace en paramètres qui combine un modèle pré-entraîné figé et quantifié en 4 bits avec des adaptateurs LoRA de bas rang.

QLoRA permet notamment d'ajuster des modèles très volumineux, jusqu'à 65 milliards de paramètres, sur un seul GPU de 48 Go, tout en conservant des performances proches d'un réglage fin complet en précision 16 bits. Le banc d'essai Vicuna, atteignant 99,3 % des performances de ChatGPT tout en ne nécessitant que 24 heures d'ajustement fin sur un seul GPU.

- **Le format 4-bit NormalFloat NF4** : un nouveau type de données optimal au sens de la théorie de l'information pour des poids suivant une distribution normale. Cette méthode repose sur la quantification par quantiles, une méthode optimale du point de vue de la théorie de l'information qui garantit que chaque intervalle de quantification reçoit un nombre égal de valeurs issues du tenseur d'entrée.
- **Double quantification** : la double quantification, pour réduire l'empreinte mémoire moyenne en quantifiant les constantes de quantification. La première étape comme des entrées pour une seconde quantification. Cette étape produit des constantes de quantification quantifiées. La seconde étapes, on utilise des nombres à virgule flottante sur 8 bits avec une taille de bloc de 256, car aucune dégradation des performances n'est observée avec une quantification sur 8 bits.
- **Optimiseurs paginés** : pour gérer les pics de consommation mémoire et éviter tout dépassement. On exploite la fonctionnalité « Unified Memory 3 » de NVIDIA, qui assure des transferts automatiques de pages entre le CPU et le GPU. Cela permet un traitement GPU sans erreur même lorsque la mémoire du GPU vient

à manquer ponctuellement.

QLoRA utilise un type de données de stockage à faible précision (généralement 4 bits dans notre cas) et un type de données de calcul (généralement BFloat16). En pratique, cela signifie que chaque fois qu'un tenseur de poids QLoRA est utilisé, nous déquantifions le tenseur en BFloat16, puis nous effectuons une multiplication matricielle en 16 bits. Nous présentons maintenant les composants de QLoRA.

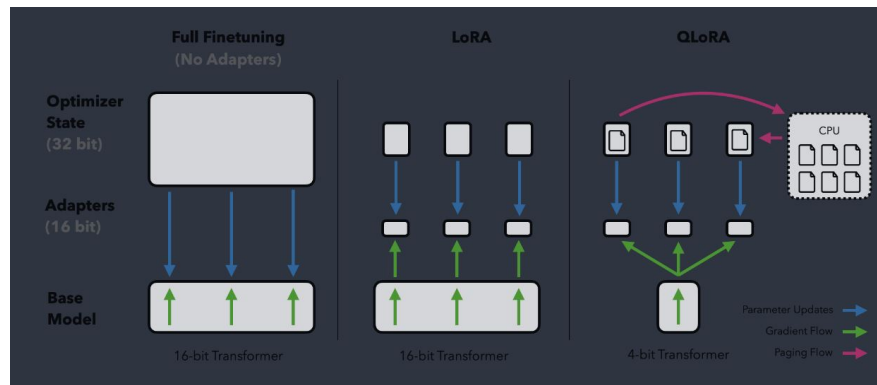


FIGURE 4.1 – QLoRA améliore LoRA en quantifiant le modèle Transformer à une précision de 4 bits.

(Low-rank Adapters) : Le réglage fin par adaptateurs de rang faible (LoRA) [28] est une méthode qui réduit les besoins en mémoire en utilisant un petit ensemble de paramètres entraîna- bles — souvent appelés adaptateurs — sans mettre à jour l’intégralité des paramètres du modèle, qui restent figés

Un point important à considérer concerne les besoins en mémoire de LoRA durant l’entraî- nement, tant en termes de nombre que de taille des adaptateurs utilisés. L’empreinte mémoire de LoRA étant très réduite, il est possible d’utiliser davantage d’adaptateurs pour améliorer les performances sans augmenter de manière significative la mémoire totale consommée.